Evolutionary Weightless Neural Networks for Network Intrusion Detection

Anonymous

Anonymous Institution

Abstract. Weightless Neural Networks (WNNs) based on Random Access Memory (RAM) neurons offer constant-time inference with submicrosecond lookup, making them attractive for edge-deployed intrusion detection systems. However, their classification performance depends on *connectivity*, which input bits each neuron observes, a problem traditionally solved by random initialization. We present an evolutionary optimization framework that jointly optimizes neuron count, address width, and connectivity through genetic algorithm, with connectivity optimization effectively performing automated feature selection intrinsic to the architecture. The network architecture is specified in a few kilobytes (neuron connections), while the trained memory occupies tens of megabytes (sparse lookup tables), comparable to traditional ML models but with $O(1)$ inference per neuron. We evaluate on three standard IDS benchmarks (UNSW-NB15, CICIDS2017, CIC-IoT-2023) using both temporal and random splits, with an eight-mode threshold calibration protocol that exposes the sensitivity of binary classifiers to decision boundary selection. Across 100+ independent runs per configuration, our approach achieves [TODO: headline numbers] outperforming Random Forest and XGBoost on the standard temporal split while maintaining sub-microsecond inference potential on FPGA.

Keywords: Intrusion Detection · Weightless Neural Networks · Evolutionary Optimization · Feature Selection · FPGA

1 Introduction

Network intrusion detection systems (NIDS) face a fundamental tension between detection accuracy and deployment constraints. Deep learning models achieve state-of-the-art accuracy but require megabytes of parameters and GPU inference, making them impractical for line-rate packet classification on edge devices. Conversely, lightweight rule-based systems operate at wire speed but lack the adaptability to detect novel attack patterns.

Weightless Neural Networks (WNNs) [2,13] offer a compelling middle ground: RAM-based neurons perform classification via direct memory lookup in $O(1)$ time, with model sizes measured in bytes rather than megabytes. Recent work [11] demonstrated WNN-based IDS achieving 98% accuracy on UNSW-NB15 using random train/test splits, which show stronger performance than temporal splits due to shared distributional characteristics between training and testing data. Additionally, neuron connectivity (which input bits each neuron observes) plays a critical role in WNN architectures but is traditionally generated randomly, where network quality depends on the initial assignment.

Our work improves on two fronts:

- *How the network is derived.* We introduce an optimization framework using genetic algorithm to discover optimal neuron configurations: count, address width, and connectivity. These optimizations effectively perform automated feature selection intrinsic to the WNN architecture.
- *How the network is evaluated.* We test the resulting networks against unseen data using temporal splits across three datasets, with a threshold calibration study that quantifies the gap between train-time and deployment-time performance.

Our contributions are:

1. **Evolved connectivity as feature selection.** Optimizing network parameters discovers which input features matter without external feature selection, with connectivity optimization providing the largest single improvement in classification performance.
2. **Compact architecture with sub-microsecond inference potential.** Our evolved WNN outperforms Random Forest and XGBoost on the standard temporal split, with a network architecture specified in a few kilobytes and $O(1)$ lookup-based inference suitable for FPGA deployment.
3. **Eight-mode threshold calibration study.** We identify threshold selection as a critical but under-reported factor in IDS evaluation, showing [TODO: X]% F1 gap between oracle and train-calibrated thresholds.
4. **Three-dataset evaluation.** Results on UNSW-NB15, CICIDS2017, and CIC-IoT-2023 demonstrate architecture generalizability across different network environments and attack types.

2 Background

2.1 RAM Neurons and Weightless Neural Networks

Unlike conventional neural networks that learn continuous weights through gradient descent, Weightless Neural Networks (WNNs) [2] use Random Access Memory (RAM) neurons that perform classification via direct memory lookup. A RAM neuron consists of 2^n memory cells, where n is the number of observed input bits, the neuron's *address width*. Given a binary input vector of length d, the neuron selects n bits through a fixed *connectivity map* $C = \{c_1, \ldots, c_n\}$ where $c_i \in \{0, \ldots, d-1\}$, concatenates them to form an n-bit address, and returns the stored value at that address. Training writes target labels to addressed cells; inference reads them in $O(1)$ time with no multiply-accumulate operations.

The fundamental mechanism enabling WNN generalization is *partial connectivity* [2,13]. A fully-connected neuron ($n = d$) memorizes every training example

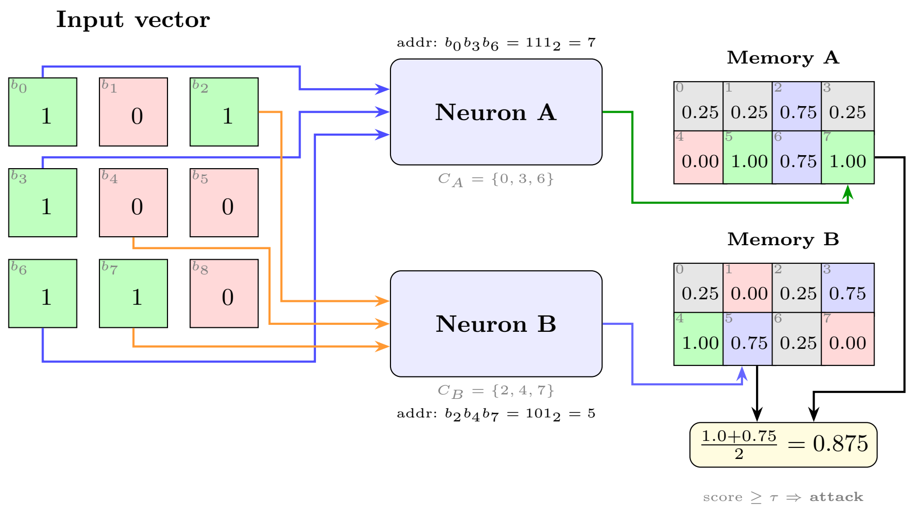

Fig. 1. RAM neuron architecture with two neurons (N=2, n=3 address bits each). Each neuron selects 3 bits from the 9-bit input via its connectivity map, forms a 3-bit address, and looks up its quad-weighted memory (2^3=8 cells). Cell colors: TRUE (1.0), WEAK_TRUE (0.75), WEAK_FALSE (0.25), FALSE (0.0). The mean score is thresholded to classify the input.

without generalizing, since each input maps to a unique address. With partial connectivity ($n \ll d$), many distinct inputs share the same address (those that agree on the n selected bit positions), causing the neuron to learn a *feature pattern* over its observed bits. An ensemble of N neurons with diverse connectivity maps provides N complementary views of the input, and their aggregated scores produce a classification decision. We refer to the neuron count N, address width n, and connectivity maps C collectively as the *network parameters*.

Quad-weighted memory. Traditional RAM neurons store binary values (TRUE/FALSE), while Probabilistic Logic Nodes (PLN) add an *undefined* state (u) for untrained cells, and Multi-valued PLN (MPLN) [13] generalize to finer granularity (e.g., 11 levels from 0.0 to 1.0 in 0.1 steps, starting at 0.5 for undefined).

We introduce *quad-weighted* (QW) memory with four states encoded in just 2 bits: FALSE (0.0), WEAK_FALSE (0.25), WEAK_TRUE (0.75), and TRUE (1.0). Unlike PLN's symmetric undefined state (0.5), QW cells default to WEAK_FALSE (0.25), biasing untrained neurons toward the majority class (normal traffic). When a cell trained as TRUE is subsequently addressed with a normal-class example, it transitions to WEAK_TRUE rather than being overwritten, preserving partial confidence from both classes. This provides smoother probability estimates than binary voting while requiring only 2 bits per cell, more compact than MPLN's multi-level representation.

2.2 Thermometer Encoding

WNNs require binary inputs, but network flow features (packet counts, byte rates, inter-arrival times) are continuous. We convert each feature to a binary vector using *thermometer encoding* [12]. Given a feature value x and k thresholds $\theta_1 < \cdots < \theta_k$, the encoding produces k bits: bit i is 1 if $x \geq \theta_i$, else 0. This preserves ordinal relationships: similar values differ by few bits, enabling RAM neurons to generalize across nearby feature values.

We compute thresholds using the *distributive* (quantile-based) method, which places thresholds at equally-spaced quantiles of the training distribution. This is robust to skewed distributions common in network traffic (e.g., packet sizes follow a heavy-tailed distribution). With k bits per feature and a feature set F, the total input width is $d = k \times |F|$ bits.

2.3 Evaluation Datasets

We evaluate on three standard IDS benchmarks, each with distinct characteristics:

UNSW-NB15 [6] contains network flow records from the Australian Centre for Cyber Security, with 42 numeric features and 10 attack categories. The standard temporal split (175K train / 82K test) separates training and testing periods, preventing data leakage. We also evaluate on a deduplicated random split (1.4M / 158K) for comparison with prior work [11].

CICIDS2017 [10] comprises five days of network traffic captured at the Canadian Institute for Cybersecurity, with 78 flow-level features and 14 attack types. We use a stratified 80/20 random split (2.3M train / 566K test), consistent with the majority of published work on this dataset.

CIC-IoT-2023 [7] captures IoT traffic from 105 heterogeneous devices with 33 attack types across 7 classes. With 46.7M records heavily skewed toward attacks (97.6%), we subsample to 1.3M records for a balanced benchmark. Only a random split is feasible (no temporal ordering in the source data).

3 Related Work

3.1 WNN-Based Intrusion Detection

The WiSARD [13] architecture pioneered RAM-based pattern recognition, and several works have applied WNNs to network security. Santiago et al. [9] demonstrated WNN-based anomaly detection with random connectivity.

The most directly related work is FWIW [11], which achieves 98% accuracy on UNSW-NB15 with a 272-byte model deployed on FPGA. FWIW uses Bloom filter-based neurons with H3 hashing and backpropagation training, with random connectivity and one threshold mode. Our work differs in three ways: (1) we evaluate on both random and temporal splits, as random splits may allow some data leakage between train and test sets; (2) we use random connectivity as a starting point and apply a genetic algorithm to optimize neuron count; and

(3) we evaluate eight threshold modes to identify which best predicts deployment performance.

BTHOWeN [12] introduced thermometer encoding with learned thresholds for WNN classification, achieving strong results on tabular benchmarks. We adopt their encoding scheme but extend it with distributive (quantile-based) thresholds and evolutionary connectivity optimization.

3.2 Machine Learning for IDS

Traditional ML approaches (Random Forest, XGBoost, SVM) achieve 85–90% F1 on UNSW-NB15's temporal split [16] but require models of 10–50 MB. Deep learning approaches (BiLSTM [1], CNN-LSTM) push accuracy higher at the cost of GPU inference requirements.

A critical methodological issue pervades the IDS literature: most published results use random train/test splits, reporting 97–99% accuracy that does not reflect deployment performance. Zoghi and Serpen [16] showed that the same models achieving 99% on random splits drop to 87% on the standard temporal split, a finding consistent with our evaluation. For CICIDS2017, the situation is similar: near-perfect random-split results contrast with the absence of temporal evaluation in the literature. Known labeling errors [4] further complicate comparisons.

3.3 FPGA-Based Network Security

FPGA acceleration for network security spans from packet-level inspection (Pigasus [15]) to ML inference (LogicNets [14], PolyLUT [3]). WNNs are particularly well-suited for FPGA deployment because RAM neurons map directly to FPGA lookup tables (LUTs) and block RAM (BRAM), requiring no multiply-accumulate units. FWIW [11] demonstrated this with sub-microsecond inference on a Virtex UltraScale+. Our work extends this direction with evolved connectivity that can further reduce resource utilization by discovering sparser, more efficient neuron configurations.

4 Architecture

4.1 Single-Cluster Binary Discriminator

Our IDS uses a *single-cluster discriminator*: N quad-weighted RAM neurons, each with b address bits, observing the same thermometer-encoded input vector $\mathbf{x} \in \{0,1\}^d$. Each neuron i has a connectivity map $C_i = \{c_1^i, \ldots, c_b^i\}$ that selects b bits from $\mathbf{x}$ to form an address. The neuron outputs the value stored at that address in its 2^b-cell memory.

Training iterates over labeled examples: for each flow, each neuron computes its address and writes the target label (0 for normal, 1 for attack) to the addressed cell using the quad-weighted update rule (conflicting writes produce weak states rather than overwriting).

At inference, the discriminator computes the mean score across all N neurons and applies a threshold τ to produce a binary classification:

$$\hat{y} = \begin{cases} 1 \text{ (attack)} & \text{if } \frac{1}{N}\sum_{i=1}^{N} \text{mem}_i[\text{addr}_i(\mathbf{x})] \geq \tau \\ 0 \text{ (normal)} & \text{otherwise} \end{cases} \tag{1}$$

This architecture is deliberately simple, a single cluster with one threshold, making the connectivity optimization problem tractable while still achieving competitive accuracy. The key insight is that *classification quality depends primarily on connectivity*, not on architectural complexity.

4.2 Phased Optimization Pipeline

We optimize the discriminator's network parameters (N, b, C) through a two-phase pipeline, where the output population of the first phase seeds the second.

Phase 1: Grid Search. Exhaustive evaluation of (N, b) configurations (e.g., $N \in \{5, 10, \ldots, 500\}$, $b \in \{4, 8, \ldots, 34\}$) with random connectivity. Each configuration is trained and evaluated, then ranked by the fitness function. The top-k configurations seed the subsequent GA population.

Phase 2: GA Neurons. A genetic algorithm optimizes neuron count N while preserving learned connections. Crossover exchanges neuron groups between parents; mutation adds or removes neurons. This phase discovers the optimal model capacity for the dataset.

The framework supports additional phases (GA bits, GA connections, tabu search refinement, and Lamarckian evolution) which are evaluated in a follow-up study.

Fitness function. Both phases rank genomes using a weighted harmonic mean of per-metric ranks within the population:

$$\text{fitness}(g) = \frac{w_{CE} + w_{F1} + w_{FPR}}{\frac{w_{CE}}{r_{CE}(g)} + \frac{w_{F1}}{r_{F1}(g)} + \frac{w_{FPR}}{r_{FPR}(g)}} \tag{2}$$

where $r_m(g)$ is genome g's rank for metric m (lower rank = better). We use w_{CE}=0.2, w_{F1}=0.3, w_{FPR}=0.4, w_{Acc}=0.1, where CE (cross-entropy) measures prediction confidence, F1 is the harmonic mean of precision and recall, and FPR (false positive rate) is the fraction of benign flows misclassified as attacks.

4.3 GPU-Accelerated Optimization

Population-based optimization requires evaluating hundreds of genome configurations per generation across hundreds of generations. We implement a Rust accelerator with Apple Metal GPU support that parallelizes both training (writing target labels to neuron memories) and evaluation (computing scores across all test examples).

The accelerator computes neuron addresses via GPU kernel dispatch ($N_{\text{neurons}} \times N_{\text{examples}}$ parallel threads), achieving throughput of $\sim$100M address computations per second on an M4 Max GPU (40 cores). This makes the full optimization pipeline practical: a complete 2-phase run (grid search + GA neurons) completes in 5 minutes on average (range 3–11 minutes) per seed on UNSW-NB15, enabling 100+ statistically independent runs.

5 Evaluation Methodology

5.1 Temporal and Random Evaluation Protocols

Evaluation protocol is a critical factor in IDS benchmarking. Random train/test splits allow training and test sets to share temporal characteristics (and, in datasets with duplicate records, near-identical flows), which can lead to higher reported accuracy. Temporal splits, where training data precedes test data chronologically, better simulate deployment conditions where the model must generalize to future traffic patterns.

We evaluate under both protocols for UNSW-NB15, and random splits for CICIDS2017 and CIC-IoT-2023:

– **Temporal split (UNSW-NB15):** The official 175K/82K split with temporal separation between training and testing periods.
– **Random splits (all datasets):** Stratified random splits, enabling comparison with prior work. UNSW-NB15 uses a deduplicated 90/10 split (1.4M/158K); CICIDS2017 uses 80/20 (2.3M/566K); CIC-IoT-2023 uses 80/20 (1.1M/268K).

5.2 Eight-Mode Threshold Calibration

Binary IDS classifiers produce a continuous score that must be thresholded to yield a classification. The choice of threshold dramatically affects F1 and FPR, yet most published work reports results under a single (often unspecified) threshold mode. We evaluate every genome under eight calibration strategies to expose this sensitivity:

1. **Train-calibrated:** Threshold swept on training-set scores (optimistic, same data used for model training).
2. **Holdout-calibrated:** Threshold swept on a held-out 25% validation split, applied to the test set (simulates having labeled calibration data from the deployment environment).
3. **Fixed 0.5:** No calibration; the midpoint threshold (distribution-agnostic baseline).
4. **Platt scaling [8]:** Logistic regression on held-out scores, learning $P(\text{attack}) = \sigma(a \cdot s + b)$.
5. **Beta calibration [5]:** Three-parameter logit model on held-out scores, more flexible than Platt for asymmetric distributions.
6. **Empirical:** Histogram-based threshold from held-out score distribution.
7. **Empirical cumulative:** CDF-based threshold selection.
8. **Oracle:** Threshold swept on the held-out validation set (not used during training or GA selection). This represents an upper bound on achievable performance, not available in deployment, but quantifies the calibration gap.

The gap between oracle and train-calibrated performance quantifies how much accuracy is "left on the table" by imperfect threshold selection, a finding we report for each dataset and configuration.

5.3 Statistical Protocol

Each configuration is run as 112 independent flows with different seeds. We report the 5% trimmed mean (removing the 6 highest and 6 lowest values, yielding 100 samples) $\pm$ standard deviation. With $n = 100$ and typical $\sigma \approx 1.4\%$, the 95% confidence interval for the mean is $\pm 0.27\%$, sufficient to distinguish meaningful differences between configurations.

K-fold cross-validation (k=5) is applied *per generation* within the GA to reduce selection noise: each genome is evaluated on 5 different training subsets, and the averaged fitness determines selection. This prevents the GA from overfitting to a particular data partition.

All reported results use the *best-fitness genome from the final optimization phase*, evaluated by the full-dataset evaluator (175K train $\rightarrow$ 82K test for UNSW-NB15) with all eight threshold modes. Each flow contributes one data point to the aggregate statistics, ensuring that reported means reflect the full distribution of outcomes.

6 Experimental Results

6.1 UNSW-NB15 Results

[TODO: Table from ids_results.md — top20-CE4F3R3 (34 flows)]

6.2 CICIDS2017 Results

[TODO: Results from 34 CICIDS2017 temporal flows once completed]

6.3 CIC-IoT-2023 Results

[TODO: Results if completed before deadline]

6.4 Phase Progression Analysis

6.5 Comparison with Baselines

[TODO: Comparison table — RF, XGBoost, BiLSTM, FWIW vs Our WNN. Separate temporal and random split results.]

6.6 Model Size Analysis

[TODO: Compute actual model sizes. N neurons $\times$ b bits $\times$ 2 bits/cell $\div$ 8 = bytes. Compare with RF (50MB), DNN (5MB), FWIW (272B).]

7 Discussion

7.1 Connectivity as Feature Selection

[TODO: The GA connection optimization phase provides the largest accuracy improvement. Analysis of which features the evolved neurons select vs. RF importance ranking.]

7.2 Threshold Sensitivity

[TODO: The 5calibration challenge. Platt/beta calibration partially closes this gap. Implications for deployment.]

7.3 FPGA Deployment Potential

[TODO: BRAM requirements for our genomes. A neuron with $b = 32$ address bits needs $2^{32} \times 2$ bits = 1 GB, too large for direct mapping. But with sparse memory (most cells empty), only non-empty entries need storage. Typical occupancy analysis. Comparison with FWIW FPGA and LogicNets.]

7.4 Limitations

[TODO: Binary classification only (multi-class as future work). Threshold sensitivity requires calibration data. Temporal split not available for CIC-IoT-2023.]

8 Related Work

8.1 WNN for Intrusion Detection

[TODO: FWIW [11], Santiago et al., BTHOWeN [12]]

8.2 ML-Based IDS

[TODO: Zoghi 2024, Kasongo 2020, survey of RF/XGBoost/DL approaches on UNSW-NB15 and CICIDS2017]

8.3 FPGA-Based IDS

[TODO: LogicNets, PolyLUT, Pigasus. Hardware acceleration landscape.]

9 Conclusion

[TODO: Summary of contributions. Key result highlight. Future work: multi-class hierarchical classification, online learning for threshold adaptation, cross-dataset transfer, FPGA implementation.]

References

1. Abdalgawad, N., Sajun, A., Kaddoura, Y., Olanrewaju, O.: Enhanced intrusion detection systems performance with UNSW-NB15 data analysis. Algorithms **17**(2), 64 (2024). https://doi.org/10.3390/a17020064, https://www.mdpi.com/1999-4893/17/2/64
2. Aleksander, I., Thomas, W.V., Bowden, P.A.: WISARD: a radical step forward in image recognition. Sensor Review **4**(3), 120–124 (1984). https://doi.org/10.1108/eb007637, https://doi.org/10.1108/eb007637
3. Andronic, M., Sherborne, T., Sherborne, L., Sherborne, J., Sherborne, D., Fraser, N.J.: PolyLUT: Learning piecewise polynomials for ultra-low-latency FPGA LUT-based inference. In: Proceedings of the International Conference on Field-Programmable Logic and Applications (FPL) (2023). https://doi.org/10.1109/FPL60245.2023.00020, https://doi.org/10.1109/FPL60245.2023.00020
4. Engelen, G., Rimmer, V., Joosen, W.: Troubleshooting an intrusion detection dataset: The CICIDS2017 case study. In: Proceedings of the IEEE Security and Privacy Workshops (SPW) (2021). https://doi.org/10.1109/SPW53761.2021.00009
5. Kull, M., Silva Filho, T., Flach, P.: Beta calibration: a well-founded and easily implemented improvement on logistic calibration for binary classifiers. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS). vol. 54, pp. 623–631. PMLR (2017)
6. Moustafa, N., Slay, J.: UNSW-NB15: A comprehensive data set for network intrusion detection systems. Military Communications and Information Systems Conference (MilCIS) (2015). https://doi.org/10.1109/MilCIS.2015.7348942, https://doi.org/10.1109/MilCIS.2015.7348942
7. Neto, E.C.P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., Ghorbani, A.A.: CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment. Sensors **23**(13) (2023). https://doi.org/10.3390/s23135941
8. Platt, J.C.: Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In: Advances in Large Margin Classifiers. pp. 61–74. MIT Press (1999)
9. Santiago, L.A.Q., Verona, L.T., Rangel, F.d.O., Firmino, F.M., Lima, P.M.V., França, F.M.G.: FPGA implementation of weightless neural networks for network intrusion detection. In: Proceedings of the IEEE International Joint Conference on Neural Networks (IJCNN) (2020). https://doi.org/10.1109/IJCNN48605.2020.9207591, https://doi.org/10.1109/IJCNN48605.2020.9207591
10. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP) (2018). https://doi.org/10.5220/0006639801080116, https://doi.org/10.5220/0006639801080116
11. Susskind, Z., Arora, A., Bacellar, A.T.L., Dutra, D.L.C., Miranda, I.D.S., Breternitz Jr., M., Lima, P.M.V., França, F.M.G., John, L.K.: FWIW: Weightless neural network inference at ultra-low cost. In: Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA) (2023). https://doi.org/10.1145/3543622.3573140, https://doi.org/10.1145/3543622.3573140
12. Susskind, Z., Arora, A., John, L.K., John, E.B., Lima, P.M.V., França, F.M.G.: BTHOWeN – binary thermometer-encoded hashed optimized weightless neural

networks. In: Proceedings of the International Conference on Parallel Architectures and Compilation Techniques (PACT) (2022). https://doi.org/10.1145/3559009.3569680, https://doi.org/10.1145/3559009.3569680
13. Teresa Bernarda Ludermir, André de Carvalho, A.P.B., de Souto, M.C.P.: Weightless neural models: A review of current and past works. In: Neural Computing Surveys 2 - ICSI Berkeley. pp. 41–61. Berkeley, USA (1999), https://www.cin.ufpe.br/~tbl/artigos/vol2_2-ncs-1999.pdf
14. Umuroglu, Y., Akhauri, Y., Fraser, N.J., Blott, M.: LogicNets: Co-designed neural networks and circuits for extreme-throughput applications. In: Proceedings of the International Conference on Field-Programmable Logic and Applications (FPL) (2020). https://doi.org/10.1109/FPL50879.2020.00065, https://doi.org/10.1109/FPL50879.2020.00065
15. Zhao, Z., Sadok, H., Atre, N., Hoe, J.C., Kim, V.S., Gibbons, P.B.: Achieving 100Gbps intrusion prevention on a single server. In: Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI) (2020), https://www.usenix.org/conference/osdi20/presentation/zhao-zhipeng
16. Zoghi, Z., Serpen, G.: Building an intrusion detection system on UNSW-NB15: Reducing the margin of error to deal with data overlap and imbalance. Concurrency and Computation: Practice and Experience (2024). https://doi.org/10.1002/cpe.8242, https://onlinelibrary.wiley.com/doi/full/10.1002/cpe.8242